

# Supplementary Material

## Learning Disentangled Robot Skills for Diverse Action Generation via Diversified Disentangled Metric Flow

Author Names Omitted for Anonymous Review. Paper-ID [2756]

### I. OVERVIEW

This document provides supplementary material and extended analysis to support the methodological details and empirical claims of the approach *Diversified Disentangled Metric Flow* (DDMF), presented in the main paper *Learning Disentangled Robot Skills for Diverse Action Generation via Diversified Disentangled Metric Flow*. This supplementary material is organized as follows: Section II presents the full mathematical derivations for the loss functions used during policy training. Section III discusses the complete DDMF training algorithm and a formal analysis of its time complexity. Section IV outlines the specific implementation details, including the simulation environments utilized for evaluation, the exact hyperparameter values, the computation protocols for the evaluation metrics, and the hardware deployment setup. Finally, Section V provides additional qualitative results demonstrating the performance of DDMF.

TABLE I: Mathematical notations.

| Variable                                   | Definition                               |
|--------------------------------------------|------------------------------------------|
| $\mathbf{s}$                               | State                                    |
| $\mathbf{a}$                               | Action                                   |
| $\mathbf{z}$                               | Latent variable                          |
| $\mathbf{x}$                               | Flow variable                            |
| $\mathcal{L}$                              | Loss                                     |
| $\mathbf{q}$                               | Pose                                     |
| $\dot{\mathbf{q}}$                         | Velocity                                 |
| $n$                                        | Number of physical factors               |
| $\omega$                                   | Parameters of the encoder network        |
| $\psi$                                     | Parameters of the flow policy network    |
| $r$                                        | Reward                                   |
| $t$                                        | Time                                     |
| $\lambda$                                  | Coefficient for balancing losses         |
| $\alpha$                                   | Temperature coefficient for entropy term |
| $\gamma$                                   | Discount variable                        |
| $\mathcal{S}$                              | State set                                |
| $\mathcal{A}$                              | Robot action set                         |
| $\mathcal{Z}$                              | Latent skill set                         |
| $\mathcal{B}$                              | Transition batch                         |
| $C$                                        | Covariance                               |
| $\mathbf{M}$                               | Binary mask                              |
| $\beta^i$                                  | Per-factor learnable weight              |
| Function                                   | Definition                               |
| $\phi_\omega(\mathbf{s})$                  | Encoder network                          |
| $\pi(\mathbf{a} \mathbf{s}, \mathbf{z})$   | Flow policy network                      |
| $\mathcal{H}(\pi(\cdot))$                  | Entropy                                  |
| $d_{\mathcal{Z}}(\mathbf{s}, \mathbf{s}')$ | Euclidean distance in latent space       |

### II. MATHEMATICAL DERIVATIONS OF LOSSES AND PROBABILITY EVALUATIONS

This section first lists the variables and mathematical notations in Table I used in our mathematical expressions and derivations, and then provides the mathematical derivations of the loss functions and the computation of probability evaluations.

#### A. Derivation of the Metric-Based Representation Loss with Lipschitz Constraint

Our approach builds upon the premise of maximizing the Wasserstein Dependency Measure (WDM) [1] between the environment states  $\mathcal{S}$  and the latent skills  $\mathcal{Z}$ .

We first introduce a score function to measure the dependency between a state and a skill, denoted as:

$$f(\mathbf{s}, \mathbf{z}) = \phi(\mathbf{s})^\top \mathbf{z} \quad (1)$$

where  $\phi : \mathcal{S} \rightarrow \mathcal{Z}$  is an encoding function.  $f(\mathbf{s}, \mathbf{z})$  outputs a scalar value representing how strongly a specific state  $\mathbf{s}$  correlates with a given latent skill  $\mathbf{z}$ . A high score indicates that the state was highly likely to be reached by executing the skill, whereas a low score indicates they are independent or mismatched.

To ensure that the score function respects the actual physical geometry of the environment,  $f(\mathbf{s}, \mathbf{z})$  must be constrained by the underlying distance metric of the state space. This is enforced by bounding the Lipschitz constant of the score function, denoted as  $\|f\|_L \leq 1$ . This constraint prevents the neural network from assigning large scores for states that are physically close. Using the constrained score function, the Kantorovich-Rubenstein duality formulates the WDM as the supremum over all valid 1-Lipschitz functions:

$$I_{\mathcal{W}}(\mathbf{s}, \mathbf{z}) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{p(\mathbf{s}, \mathbf{z})}[f(\mathbf{s}, \mathbf{z})] - \mathbb{E}_{p(\mathbf{s})p(\mathbf{z})}[f(\mathbf{s}, \mathbf{z})] \quad (2)$$

Assuming a zero-mean prior distribution for the skills, where  $\mathbb{E}_{p(\mathbf{z})}[\mathbf{z}] = 0$ , we cancel the second expectation in Eq. (2). Then, we decompose the score over a trajectory of states  $\tau$  with the length  $T$ , formulated as  $\sum_{t=0}^{T-1} (\phi(\mathbf{s}_{t+1}) - \phi(\mathbf{s}_t))$ . This yields the trajectory-level maximization objective of WDM:

$$I_{\mathcal{W}}(\tau, \mathbf{z}) \approx \sup_{\|f\|_L \leq 1} \mathbb{E}_{p(\tau), \mathbf{z}} \left[ \sum_{t=0}^{T-1} (\phi(\mathbf{s}_{t+1}) - \phi(\mathbf{s}_t))^\top \mathbf{z} \right] \quad (3)$$

Then, we define our state-space metric as the temporal distance, which is the minimum number of environment steps required to transition between two states. Under this

metric, the global Lipschitz constraint is mathematically equivalent to bounding the Euclidean distance of the latent embeddings strictly for adjacent state pairs  $(\mathbf{s}, \mathbf{s}')$ . Thus, the final constrained maximization objective is formulated as:

$$\sup_{\phi} \mathbb{E}_{p(\tau, \mathbf{z})} \left[ \sum_{t=0}^{T-1} (\phi(\mathbf{s}_{t+1}) - \phi(\mathbf{s}_t))^{\top} \mathbf{z} \right] \quad (4)$$

$$\text{s.t. } \|\phi(\mathbf{s}) - \phi(\mathbf{s}')\|_2 \leq 1 \quad \forall (\mathbf{s}, \mathbf{s}')$$

To optimize Eq. (4) via gradient descent, we update the encoder network  $\phi_{\omega}(\mathbf{s})$ . We introduce the adjacent-state distance constraint as a Lipschitz penalty using Lagrangian relaxation, yielding our final metric-based representation loss  $\mathcal{L}_{\text{rep}}$  with the Lipschitz penalty  $\mathcal{L}_{\text{Lip}}$ :

$$\mathcal{L}_{\text{rep}} + \mathcal{L}_{\text{Lip}} = \mathbb{E}_{p(\tau)} \left[ -(\phi_{\omega}(\mathbf{s}') - \phi(\mathbf{s}))^{\top} \mathbf{z} + \max(0, \|\phi(\mathbf{s}) - \phi(\mathbf{s}')\|_2 - 1) \right] \quad (5)$$

Minimizing this loss shapes the latent space such that temporal transitions in the physical environment correspond to the transitions in the latent skill space.

In practice, we additionally scale each factor  $i$ 's contribution to  $\mathcal{L}_{\text{rep}}$  by a per-factor learnable weight  $\beta^i$  [2], yielding the practical form of  $\mathcal{L}_{\text{rep}}$  given in the main text. The weight is computed by a neural network, which predicts each factor's one-step transition given the current state  $\mathbf{s}_t$ . The factors more predictable given  $\mathbf{s}_t$  are assigned more weight relative to factors with high inherent stochasticity.

### B. Derivation of the Log-Likelihood of the Generated Action from Flow Policy

We derive the computation of the log-likelihood of the action generated by the flow policy,  $\log \pi_{\psi}(\mathbf{a} | \mathbf{s}, \mathbf{z})$ , which is utilized in the entropy objective.

Because Conditional Continuous Normalizing Flow (CCNF) generates actions by continuously transforming a sampled noise distribution into a complex target distribution, the computation of the likelihood of a generated action using standard closed-form probability density functions is intractable. Therefore, we evaluate it by tracking how the probability density of a sample evolves as it undergoes this continuous transformation.

Geometrically, the neural network modeling the flow policy acts as a velocity field that guides this transformation process by stretching or compressing the continuous action space over time. To evaluate the probability density of a specific action point, we consider an infinitesimally small region enclosing it. The probability density at this point is fundamentally defined as the probability mass contained within this region divided by its spatial volume. By the principle of conservation of probability mass, as this region flows through the velocity field, its total probability mass remains constant. Therefore, to track the changing probability density of the sample, we only need to measure the change in the region's volume.

Mathematically, the change of region's volume under transformation  $T$  in the velocity field is given by the determinant of transformation function  $T$ 's Jacobian matrix [3].

To compute this change, we first represent the transformation function  $T$  as:

$$T(\mathbf{x}_t) = \mathbf{x}_t + v_{\psi}(\mathbf{x}_t, t | \mathbf{s}, \mathbf{z}) dt \quad (6)$$

Given this transformation, we compute the Jacobian matrix of  $T$  with respect to the current state  $\mathbf{x}_t$ :

$$\frac{\partial T(\mathbf{x}_t)}{\partial \mathbf{x}_t} = \mathbf{I} + \frac{\partial v_{\psi}(\mathbf{x}_t, t | \mathbf{s}, \mathbf{z})}{\partial \mathbf{x}_t} dt \quad (7)$$

where  $\mathbf{I}$  is the identity matrix. Then, we compute the volume change of the region as the determinant of the Jacobian in Eq. (7):

$$\det \left| \frac{\partial T(\mathbf{x}_t)}{\partial \mathbf{x}_t} \right| = \det \left| \mathbf{I} + \frac{\partial v_{\psi}}{\partial \mathbf{x}_t} dt \right| \quad (8)$$

$$= 1 + \text{Tr} \left( \frac{\partial v_{\psi}}{\partial \mathbf{x}_t} \right) dt + \mathcal{O}(dt^2)$$

where  $\text{Tr} \left( \frac{\partial v_{\psi}}{\partial \mathbf{x}_t} \right)$  is the trace of the Jacobian matrix of the velocity field with respect to the flow state  $\mathbf{x}_t$ .

By the conservation of probability mass, the updated probability density after step  $dt$  is inversely proportional to this volume change, meaning  $p(\mathbf{x}_{t+dt} | \mathbf{s}, \mathbf{z}) = \frac{p(\mathbf{x}_t | \mathbf{s}, \mathbf{z})}{\det \left| \frac{\partial T(\mathbf{x}_t)}{\partial \mathbf{x}_t} \right|}$ .

Taking the natural logarithm of this inverse relationship and substituting the determinant from Eq. (8) yields the discrete log-density update rule:

$$\log p(\mathbf{x}_{t+dt} | \mathbf{s}, \mathbf{z}) - \log p(\mathbf{x}_t | \mathbf{s}, \mathbf{z}) \quad (9)$$

$$= -\log \left( 1 + \text{Tr} \left( \frac{\partial v_{\psi}}{\partial \mathbf{x}_t} \right) dt \right)$$

We then apply the first-order Maclaurin series expansion for the natural logarithm, formulated as  $\log(1 + y) \approx y$  for small  $y$ , to simplify the right-hand side of Eq. (9), yielding:

$$-\log \left( 1 + \text{Tr} \left( \frac{\partial v_{\psi}}{\partial \mathbf{x}_t} \right) dt \right) \approx -\text{Tr} \left( \frac{\partial v_{\psi}}{\partial \mathbf{x}_t} \right) dt \quad (10)$$

We substitute the approximation from Eq. (10) back into Eq. (9). Then, we divide both sides of Eq. (9) by  $dt$ , and take the limit as  $dt \rightarrow 0$ . The instantaneous rate of change of the log-probability is simplified to the divergence of the conditional velocity field:

$$\frac{\partial \log p(\mathbf{x}_t, t | \mathbf{s}, \mathbf{z})}{\partial t} = -\nabla_{\mathbf{x}_t} \cdot v_{\psi}(\mathbf{x}_t, t | \mathbf{s}, \mathbf{z}) \quad (11)$$

$$= -\text{Tr} \left( \frac{\partial v_{\psi}}{\partial \mathbf{x}_t} \right)$$

We integrate the instantaneous rate of change of  $\mathbf{x}_t$ , defined in Eq. (11), over the continuous generation process from  $t = 0$  to  $t = 1$ . Let the initial state be the sampled noise  $\mathbf{x}_0 = \xi$  and the terminal state be a base action  $\mathbf{x}_1 = \mathbf{a}_{\text{base}}$ , we formulate the exact log-likelihood of  $\mathbf{a}_{\text{base}}$  as:

$$\log \pi_{\psi}(\mathbf{a}_{\text{base}} | \mathbf{s}, \mathbf{z}) = \log p(\xi) - \int_0^1 \text{Tr} \left( \frac{\partial v_{\psi}}{\partial \mathbf{x}_t} \right) dt \quad (12)$$

In practice, evaluating Eq. (12) requires integrating the trace of the Jacobian of the velocity field along the entire generation trajectory, which is computationally prohibitive to backpropagate through at the scale required for online reinforcement learning. Following PolicyFlow [4], we therefore adopt a tractable approximation for policy optimization

instead, as described in the main text; this approximation, rather than Eq. (12), is what is implemented in Algorithm 1.

### III. TRAINING ALGORITHM AND COMPLEXITY ANALYSIS OF DDMF

In this section, we first present the training algorithm of our DDMF in Algorithm 1. Then, we provide the analysis of computation complexity of our algorithm.

#### A. Training Algorithm of DDMF

The algorithm designed to train DDMF is presented in Algorithm 1. After the initialization on Line 1 and Line 2, we first collect a transition buffer from Line 4 to Line 8. Then, in each iteration of the network update loop, we first sample a mini-batch of transition on Line 10. Then, from Line 12 to Line 17, we compute the losses for training encoder networks from each sample in the batch. We compute the final loss for encoders on Line 18, and update the encoders' parameters on Line 19. We start the computation of the loss for flow policy learning on Line 20. Specifically, we compute the log-likelihood of the generated action on Line 26, and we get the flow policy learning loss on Line 27. Finally, we update the parameters of flow policy network on Line 28.

#### B. Time Complexity Analysis of the DDMF Algorithm

In this section, we analyze the time complexity of the joint training algorithm detailed in Algorithm 1 to assess the scalability and efficiency of DDMF. Let  $d$  denote the dimensionality of the latent skill space  $\mathcal{Z}$ , and let  $|\mathcal{A}|$  denote the dimensionality of the continuous action space. We define  $\mathcal{O}(T_\phi)$  as the time complexity for a single forward pass through the skill encoder network  $\phi_\omega$ . Similarly, we define  $\mathcal{O}(T_v)$  as the time complexity for a single forward pass through the velocity field network  $v_\psi$ . The algorithm proceeds over  $N_{\text{iteration}}$  training iterations, with each iteration partitioned into a transition collection phase and a network optimization phase.

We first analyze the time complexity of transition collection within a single training iteration. Starting from Line 6, the robot interacts with the environment for  $N_{\text{steps}}$ . In each step, we generate an action via the flow policy, requiring the transformation from random noise to a target action. We bound this action generation process by  $\mathcal{O}(T_{\text{act}})$ . The execution of the action and the update of the state on Line 7 incur an environment transition cost of  $\mathcal{O}(T_{\text{env}})$ . Therefore, the total time complexity for the transition collection phase per iteration is:

$$\mathcal{O}(N_{\text{steps}} \cdot (T_{\text{act}} + T_{\text{env}})) \quad (13)$$

Next, we analyze the time complexity of the encoder network training following the transition collection phase. The encoder networks are updated for  $N_{\text{update}}$  steps. Within each encoder update iteration, a mini-batch of size  $|\mathcal{B}|$  is sampled on Line 10. For each sample in  $\mathcal{B}$ , computing the latent transition requires passing the states through the encoder, taking  $\mathcal{O}(|\mathcal{B}| \cdot T_\phi)$  time. Then, the computation of the CFDR loss from Line 14 to Line 15 involves the computation of the outer product matrix and the Hadamard

product between the binary mask and the outer product matrix. This process is governed by the dimension of the latent space  $d$ , yielding a complexity of  $\mathcal{O}(|\mathcal{B}| \cdot d^2)$ . Thus, the overall time complexity for the encoder update per gradient step is formulated as:

$$\mathcal{O}(|\mathcal{B}| \cdot (T_\phi + d^2)) \quad (14)$$

To update the flow policy, we generate actions conditioned on the sampled batch of states. This continuous generation process is numerically integrated using a 2nd-order Runge-Kutta method with  $K$  steps, from Line 20 to Line 23. Because each integration step requires two evaluations of the velocity field  $v_\psi$ , the total integration cost is  $\mathcal{O}(|\mathcal{B}| \cdot K \cdot T_v)$ . Following the integration, the Gaussian noise addition (Line 24), the tangent squash operation (Line 25), and the action log-likelihood computation (Line 26) each require element-wise operations that scale linearly with the action dimension  $|\mathcal{A}|$ , taking  $\mathcal{O}(|\mathcal{B}| \cdot |\mathcal{A}|)$ . Thus, the time complexity for the flow policy update per gradient step is:

$$\mathcal{O}(|\mathcal{B}| \cdot (K \cdot T_v + |\mathcal{A}|)) \quad (15)$$

Combining the transition collection and network optimization phases across all iterations, the total asymptotic time complexity of the DDMF training algorithm is:

$$\mathcal{O}\left(N_{\text{iteration}} \cdot \left[ N_{\text{steps}}(T_{\text{act}} + T_{\text{env}}) + N_{\text{update}} \cdot |\mathcal{B}|(T_\phi + d^2 + K \cdot T_v + |\mathcal{A}|) \right] \right) \quad (16)$$

### IV. ADDITIONAL IMPLEMENTATION AND EVALUATION DETAILS

In this section, we provide comprehensive details regarding the simulation environments, network architectures, and hyperparameters utilized for training DDMF.

#### A. Simulation Environment Design

To evaluate the capability of DDMF to discover structurally disentangled and highly diverse skills, we conduct experiments across two distinct continuous control domains: manipulation in the Franka Kitchen [5] and multi-directional locomotion with the Ant robot [6]. Both environments are simulated using the MuJoCo [7] physics engine.

*a) Franka Kitchen Environment:* The Franka Kitchen environment is a complex, high-dimensional manipulation benchmark featuring a 9-DoF Franka Emika Panda robot arm situated in a kitchen setting containing various interactive objects. The environment is designed to evaluate the discovery of diverse manipulation skills without task-specific reward engineering.

Specifically, we evaluate DDMF across 7 distinct manipulation tasks, each requiring unique physical interactions with a specific object:

- **Microwave:** Grasping the handle and pulling the microwave door to a fully open angle.
- **Kettle:** Moving the kettle across the stove from its initial position to a designated target location.
- **Bottom Burner:** Rotating the control knob associated with the bottom burner to its active position.

---

**Algorithm 1:** Joint Training of Encoder Networks and Flow Policy Network

---

```
1 Initialize  $\pi_\psi(\mathbf{a}|\mathbf{s}, \mathbf{z})$ ,  $\phi_\omega(\mathbf{s})$ 
2 Initialize transition buffer  $\mathcal{D} \leftarrow \emptyset$ 
3 for  $i = 1, 2, \dots, N_{\text{iteration}}$  do
4    $\mathbf{s}_1 = [\mathbf{s}_1^1, \dots, \mathbf{s}_1^n] \leftarrow \text{reset environment}$ 
5   Sample factorized latent skills  $\mathbf{z}_i = [\mathbf{z}_i^1, \dots, \mathbf{z}_i^n] \sim p(\mathbf{z}_i)$ 
6   for  $t = 1, 2, \dots, N_{\text{steps}}$  do
7      $\mathbf{a} \leftarrow \pi_\psi(\cdot | \mathbf{s}_t, \mathbf{z}_i)$ ; execute  $\mathbf{a}$ , observe  $\mathbf{s}_{t+1}$ 
8      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}, \mathbf{s}_{t+1}, \mathbf{z}_i)\}$ ;  $\mathbf{s}_t \leftarrow \mathbf{s}_{t+1}$ 
9   for  $j = 1, 2, \dots, N_{\text{update}}$  do
10    Sample mini-batch  $\mathcal{B} = \{(\mathbf{s}, \mathbf{a}, \mathbf{s}', \mathbf{z})\} \sim \mathcal{D}$ 
11     $\mathcal{L}_{\text{rep}}, \mathcal{L}_{\text{CFDR}}, \mathcal{L}_{\text{Lip}} \leftarrow 0$ 
12    for  $(\mathbf{s}, \mathbf{a}, \mathbf{s}', \mathbf{z}) \in \mathcal{B}$  do
13       $\Delta \mathbf{z} \leftarrow \phi_\omega(\mathbf{s}') - \phi_\omega(\mathbf{s})$ ;  $r_{\text{int}} \leftarrow \Delta \mathbf{z}^\top \mathbf{z}$ 
14       $\Delta \bar{\mathbf{z}} \leftarrow \Delta \mathbf{z} - \mathbb{E}_{\mathcal{B}}[\Delta \mathbf{z}]$ ;  $C \leftarrow \Delta \bar{\mathbf{z}} \Delta \bar{\mathbf{z}}^\top$ 
15       $\mathcal{L}'_{\text{CFDR}} \leftarrow \frac{1}{\|\mathbf{M}\|_1} (\mathbf{M} \odot C^{\odot 2})$ ;  $\mathcal{L}'_{\text{Lip}} \leftarrow \max(0, d_{\mathcal{Z}}(\mathbf{s}, \mathbf{s}') - 1)$ 
16       $\mathcal{L}_{\text{CFDR}} \leftarrow \mathcal{L}_{\text{CFDR}} + \mathcal{L}'_{\text{CFDR}}$ ;
17       $\mathcal{L}_{\text{rep}} \leftarrow \mathcal{L}_{\text{rep}} - r_{\text{int}}$ ;  $\mathcal{L}_{\text{Lip}} \leftarrow \mathcal{L}_{\text{Lip}} + \mathcal{L}'_{\text{Lip}}$ 
18     $\mathcal{L}(\omega) \leftarrow \frac{1}{|\mathcal{B}|} (\mathcal{L}_{\text{rep}} + \lambda_1 \mathcal{L}_{\text{CFDR}} + \lambda_2 \mathcal{L}_{\text{Lip}})$ 
19    Update  $\omega$  with  $\mathcal{L}(\omega)$ 
20    Sample  $\mathbf{x}_0 \sim \mathcal{N}(0, \mathbf{I})$  and  $(\mathbf{s}, \mathbf{z}) \sim \mathcal{B}$ 
21    for  $k = 0, \dots, K - 1$  do
22       $t_k = k/K$ ,  $\Delta t = 1/K$ ;  $\mathbf{x}_{k+1/2} \leftarrow \mathbf{x}_k + \frac{\Delta t}{2} v_\psi(\mathbf{x}_k, t_k | \mathbf{s}, \mathbf{z})$ 
23       $\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k + \Delta t v_\psi(\mathbf{x}_{k+1/2}, t_k + \frac{\Delta t}{2} | \mathbf{s}, \mathbf{z})$ 
24       $\mathbf{a}_{\text{base}} \leftarrow \mathbf{x}_K$ ;  $\delta \sim \mathcal{N}(0, \sigma_\psi)$ ;  $\mathbf{u} \leftarrow \mathbf{a}_{\text{base}} + \delta$ 
25       $\mathbf{a} \leftarrow \tanh(\mathbf{u})$ 
26       $\log \pi_\psi(\mathbf{a} | \mathbf{s}, \mathbf{z}) \leftarrow \log p(\delta) - \sum_d \log(1 - \tanh^2(\mathbf{u}_d))$ 
27      Estimate accumulated return  $R_{\text{int}}$  from  $r_{\text{int}}$ ;  $\mathcal{L}(\psi) \leftarrow \mathbb{E}_{\mathbf{s}, \mathbf{z} \sim \mathcal{B}} [\alpha \log \pi_\psi(\mathbf{a} | \mathbf{s}, \mathbf{z}) - R_{\text{int}}]$ 
28      Update  $\psi$  with  $\mathcal{L}(\psi)$ 
29 return  $\pi_\psi(\cdot)$ 
```

---

- **Top Burner:** Rotating the control knob associated with the top burner to its active position.
- **Slide Cabinet:** Horizontally translating the sliding cabinet door.
- **Hinge Cabinet:** Pulling the cabinet door outward along its hinge.
- **Light Switch:** Manipulating the toggle switch of the light to push it into the ‘on’ state.

The continuous action space  $\mathcal{A} \in \mathbb{R}^9$  corresponds to the joint velocity commands for the robot arm and gripper. The state space  $\mathcal{S} \in \mathbb{R}^{60}$  consists of the robot’s proprioceptive information concatenated with the spatial configurations, including pose and velocities, of all interactable objects in the scene. Together, these settings form a comprehensive benchmark that challenges the learned policy to acquire disentangled manipulation skills while remaining strictly invariant to the states of the other objects.

*b) Ant Locomotion Environment:* To evaluate DDMF in a contact-rich continuous control setting, we utilize the quadrupedal Ant environment. The Ant is modeled as a 3D robot featuring a central torso and four independent legs, each possessing two actuated joints. This environment is

designed to evaluate the unsupervised discovery of diverse locomotion behaviors. Specifically, the primary objective in this domain is to discover and formalize distinct directional locomotion skills. These learned skills correspond to maximizing state-space distance over time. Consequently, the Ant robot learns to navigate effectively by discovering two distinct types of movement:

- **Linear Translation:** Moving in entirely distinct coordinate directions across the  $xy$ -plane.
- **Angular Rotation:** Rotating the torso to achieve diverse yaw orientations.

The continuous action space  $\mathcal{A} \in \mathbb{R}^8$  corresponds to the torque commands applied to the 8 actuated leg joints. We augment the standard observation space, resulting in a state space  $\mathcal{S} \in \mathbb{R}^{32}$ . This comprises the robot’s global Cartesian coordinates, its 3D orientation, alongside the standard linear and angular velocities, and the kinematic states of all joint hinges. In this domain, a learned disentangled latent skill space  $\mathcal{Z}$  should allow the conditional flow policy to map distinct latent skills into independent linear or angular movements.

### B. State Space Factorization Design

The DDMF approach explicitly structures disentangled representations of discovered skills, ensuring that a single latent skill dimension exclusively controls a specific physical factor in the environment. To enforce this mathematically via the Cross-Factor Disentanglement Regularization (CFDR) loss and the metric-regularized objective, we explicitly factorize the continuous state space  $\mathcal{S}$  into independent subspaces. This factorization allows the skill encoder  $\phi_\omega$  to track independent state transitions and guides the conditional flow policy to map each latent skill to distinct physical robot behaviors. The design of this state factorization is specified by the distinct challenges of each simulated evaluation environment.

*a) State Factorization in Franka Kitchen:* In the Franka Kitchen environment, we explicitly factorize the state vector  $\mathbf{s}$  into 5 disjoint sub-spaces, where each sub-space isolates the physical state of a specific environmental component. The factorization is defined as follows:

- **Factor 0 (Robot Arm & Gripper):**  $\mathbf{s}[0 : 18]$  captures the proprioceptive state of the robot, comprising 9 joint positions and 9 joint velocities.
- **Factor 1 (Switches & Dials):**  $\mathbf{s}[18 : 38]$  captures the actuation states of the bottom burner, top burner, and light switch.
- **Factor 2 (Cabinets):**  $\mathbf{s}[38 : 44]$  isolates the kinematic configurations of the sliding cabinet and the hinge cabinet.
- **Factor 3 (Microwave):**  $\mathbf{s}[44 : 46]$  tracks the hinge angle and velocity of the microwave door.
- **Factor 4 (Kettle):**  $\mathbf{s}[46 : 60]$  captures the spatial pose and velocity of the kettle.

*b) State Factorization in Ant Locomotion:* In the Ant environment, we factorize the state vector into 2 specific factors to decouple directional movement from rotation:

- **Factor 0 (Linear Translation):**  $\mathbf{s}[0 : 2]$  captures the global  $x$  and  $y$  Cartesian coordinates of the robot's torso.
- **Factor 1 (Angular Rotation):**  $\mathbf{s}[2 : 3]$  captures the global yaw angle of the robot torso.

The remaining 29 dimensions of the state vector,  $\mathbf{s}[3 : 32]$ , which include the raw orientation quaternions, torso height, roll and pitch angles, linear and angular velocities, and all joint positions and velocities, are preserved as essential proprioceptive information for the policy and the skill encoder.

*c) Factorization in the Entangled Ablations:* For the SUSF w/o Fact. and DDMF w/o Fact. ablations discussed in the main text, no factorization is applied. We treat the full state vector  $\mathbf{s}$  as a single, undivided factor, collapsing the latent skill space to a single entangled latent variable. We train both methods by running Algorithm 1 with  $n = 1$ . All other hyperparameters remain unchanged.

### C. Hyperparameter Values

The hyperparameters utilized to train the DDMF are summarized in Table II. We maintain a consistent configuration

across all domains to demonstrate the robustness of our approach, with only minimal environment-specific adjustments to the network learning rates and latent dimensions to account for the varying complexity of the tasks.

### D. Computation of Evaluation Metrics

To quantify the performance of the DDMF framework against baseline methods, we employ two primary evaluation metrics: the DCI (Disentanglement, Completeness, and Informativeness) framework [8] to assess the quality of the learned latent skill space, and State Coverage to measure the diversity of the generated robot behaviors.

*a) DCI Metric and Importance Matrix:* The DCI metric evaluates the statistical relationship between the learned latent skills  $\mathbf{z} \in \mathbb{R}^d$  and the ground-truth physical factors defined in Section IV-B, denoted as  $\mathbf{s} = [\mathbf{s}^1, \dots, \mathbf{s}^n]$ .

We first compute an Importance Matrix  $R \in \mathbb{R}^{d \times n}$ . Each entry  $R_{i,j}$  represents the relative importance of the  $i$ -th dimension of the learned composite skill vector  $\mathbf{z}$  in predicting the  $j$ -th ground-truth physical state factor  $\mathbf{s}^j$ . Specifically, the Importance Matrix  $R$  is constructed by training a Gradient Boosted Tree regressor, represented as  $f_{\text{reg}} : \mathcal{Z} \rightarrow \mathcal{S}$ , to predict each ground-truth state factor  $\mathbf{s}^j$  from the latent representation  $\mathbf{z}$ . The entry  $R_{i,j}$  is quantified by calculating the normalized, weighted decrease in variance achieved by node splits utilizing latent dimension  $\mathbf{z}_i$  across all trees in the regression ensemble [8], [9].

*b) Computation of Disentanglement:* This measures whether each learned skill  $\mathbf{z}^i$  captures at most one ground-truth state factor. First, we compute the probability that skill  $\mathbf{z}^i$  is used to predict factor  $\mathbf{s}^j$  by normalizing the rows of the importance matrix:  $P_{ij} = \frac{R_{ij}}{\sum_{k=1}^n R_{ik}}$ . Then, we compute the entropy of this distribution for each skill  $\mathbf{z}^i$ , which quantifies the extent to which a learned skill exclusively influences a single physical factor. We compute the disentanglement score  $D_i$  for a single skill as:

$$D_i = 1 - \mathcal{H}_n(P_i) = 1 + \sum_{j=1}^n P_{ij} \log_n(P_{ij}) \quad (17)$$

If  $\mathbf{z}^i$  exclusively predicts a single state factor, the entropy  $\mathcal{H}_n(P_i)$  is zero and  $D_i = 1$ . The total Disentanglement score is the weighted average of  $D_i$ , where the weights  $\rho_i = \frac{\sum_{j=1}^n R_{ij}}{\sum_{i=1}^d \sum_{j=1}^n R_{ij}}$  represent the relative predictive importance of skill  $\mathbf{z}^i$ . Given the computed weights, we formulate the total Disentanglement score as:

$$D = \sum_{i=1}^d \rho_i D_i \quad (18)$$

*c) Computation of Completeness:* As the inverse of disentanglement, this measures whether each ground-truth state factor  $\mathbf{s}^j$  is captured by at most learned skill. We calculate the probability that factor  $\mathbf{s}^j$  is predicted by skill  $\mathbf{z}^i$  by normalizing the columns of the importance matrix:  $\tilde{P}_{ij} = \frac{R_{ij}}{\sum_{k=1}^d R_{kj}}$ . The completeness score  $C_j$  of the  $j$ -th skill for a single ground-truth factor is computed via base- $d$

TABLE II: Hyperparameters for the DDMF framework.

| Hyperparameter                                   | Value                                              |
|--------------------------------------------------|----------------------------------------------------|
| Mini-batch Size ( $ \mathcal{B} $ )              | 256                                                |
| Factor Weight $\beta^i$                          | Computed from a learned distance predictor network |
| Learning Rate of Flow Policy $\pi_\psi$          | 1e-4                                               |
| Learning Rate of Skill Encoder $\phi_\omega$     | Ant: 1e-4, Franka Kitchen: 5e-5                    |
| Learning Rate of Critic Network                  | Ant: 3e-4, Franka Kitchen: 1e-4                    |
| Learning Rate of Entropy $\alpha$                | 1e-4                                               |
| Gradient Clipping                                | 100.0                                              |
| Discount Factor ( $\gamma$ )                     | 0.99                                               |
| Target Entropy Coefficient                       | 1.0                                                |
| Reward Scale Factor                              | 1.0                                                |
| ODE Integration Algorithm                        | Midpoint (RK2)                                     |
| Integration Steps ( $K$ )                        | 10                                                 |
| Flow Network Hidden Layers                       | [256, 256, 256]                                    |
| Flow Network Activations                         | ReLU (Tanh output)                                 |
| Timestep Embedding Dimension                     | 16                                                 |
| Flow Target Exponential Moving Average Rate      | 0.995                                              |
| Initial Exploration $\sigma$                     | 1.0                                                |
| Log Exploration $\sigma$ Bounds                  | [-5.0, 4.0]                                        |
| Latent Skill Dimensions                          | Ant: [2, 1], Franka Kitchen: [2, 2, 2, 2]          |
| CFDR Balancing Coefficient ( $\lambda_1$ )       | 1.0                                                |
| Lipschitz Constraint Coefficient ( $\lambda_2$ ) | Initialized value: 3000.0, Slack: 1e-6             |
| Encoder Network Hidden Layers                    | [256, 256, 256]                                    |
| Encoder Network Activations                      | ELU (Linear output)                                |

entropy:

$$C_j = 1 - H_d(\tilde{P}_j) = 1 + \sum_{i=1}^d \tilde{P}_{ij} \log_d(\tilde{P}_{ij}) \quad (19)$$

The total Completeness score is the weighted average of  $C_j$ , where the weights  $\tau_j = \frac{\sum_{i=1}^d R_{ij}}{\sum_{i'=1}^d \sum_{j'=1}^n R_{i'j'}}$  denote the relative importance of state factor  $s^j$ . Given the computed weights, we formulate the total Completeness score as:

$$C = \sum_{j=1}^n \tau_j C_j \quad (20)$$

*d) Computation of Informativeness:* This measures how well the learned representation  $\mathbf{z}$  can predict the ground-truth factors of the composite state  $\mathbf{s}$ . Let  $E_j$  denote the predictive performance metric for the  $j$ -th factor, evaluated over a test dataset of  $N$  transition samples. Let  $\mathbf{s}^{j,m}$  represent the true  $j$ -th state factor for the  $m$ -th sample in the dataset. Given the model predictions  $\hat{\mathbf{s}}^{j,m} = [f_{\text{reg}}(\mathbf{z}^m)]^j$ , we compute the  $R^2$  coefficient of determination  $E_j$  as:

$$E_j = 1 - \frac{\sum_{m=1}^N \|\mathbf{s}^{j,m} - \hat{\mathbf{s}}^{j,m}\|_2^2}{\sum_{m=1}^N \|\mathbf{s}^{j,m} - \bar{\mathbf{s}}^j\|_2^2} \quad (21)$$

where  $\bar{\mathbf{s}}^j = \frac{1}{N} \sum_{m=1}^N \mathbf{s}^{j,m}$  is the empirical mean of the true state factor across the dataset. The overall Informativeness is then defined as the average prediction performance across all  $n$  factors:

$$I = \frac{1}{n} \sum_{j=1}^n E_j \quad (22)$$

*e) Computation of State Coverage:* To explicitly quantify the diversity of flow policy generated actions in Franka Kitchen, we compute the State Coverage by tracking the number of unique environmental states visited during an evaluation episode.

Let the full continuous observation at timestep  $t$  be denoted as  $\mathbf{s}_t \in \mathbb{R}^{60}$ . We isolate the states of the manipulable objects by applying a predefined permutation function  $P(\cdot)$  to group the relevant semantic factors. We discard the state dimensions correspond to the robot's internal proprioception, ensuring the coverage metric reflects true environmental manipulation only. We denote  $\mathbf{o}_t \in \mathbb{R}^{42}$  as the isolated object state:

$$\mathbf{o}_t = P(\mathbf{s}_t)[18 : 60] \quad (23)$$

To measure coverage in a continuous domain, we discretize the isolated physical state into a grid space. Each dimension of  $\mathbf{o}_t$  is rounded to a single decimal place, partitioning the continuous space into discrete bins of width 0.1. Let  $f(\cdot)$  denote this discretization function:

$$\hat{\mathbf{o}}_t = f(\mathbf{o}_t) = \text{round}(\mathbf{o}_t, 1) \quad (24)$$

where  $\hat{\mathbf{o}}_t$  represents the rounded physical state of objects.

Throughout an evaluation episode of length  $T = 10000$  steps, we continuously accumulate the set of unique discretized states visited by the policy, denoted as  $\mathcal{U}$ :

$$\mathcal{U} = \bigcup_{t=1}^T \{\hat{\mathbf{o}}_t\} \quad (25)$$

The final State Coverage metric is defined as the cardinality of this set, representing the absolute volume of unique, discrete object configurations the DDMF policy successfully discovered and reached:

$$\text{Coverage} = |\mathcal{U}| \quad (26)$$

#### E. Hardware Deployment Setup

To validate the real-world applicability of the locomotion skills discovered by the DDMF, we perform hardware experiments utilizing the Unitree Go2 quadruped robot. It is

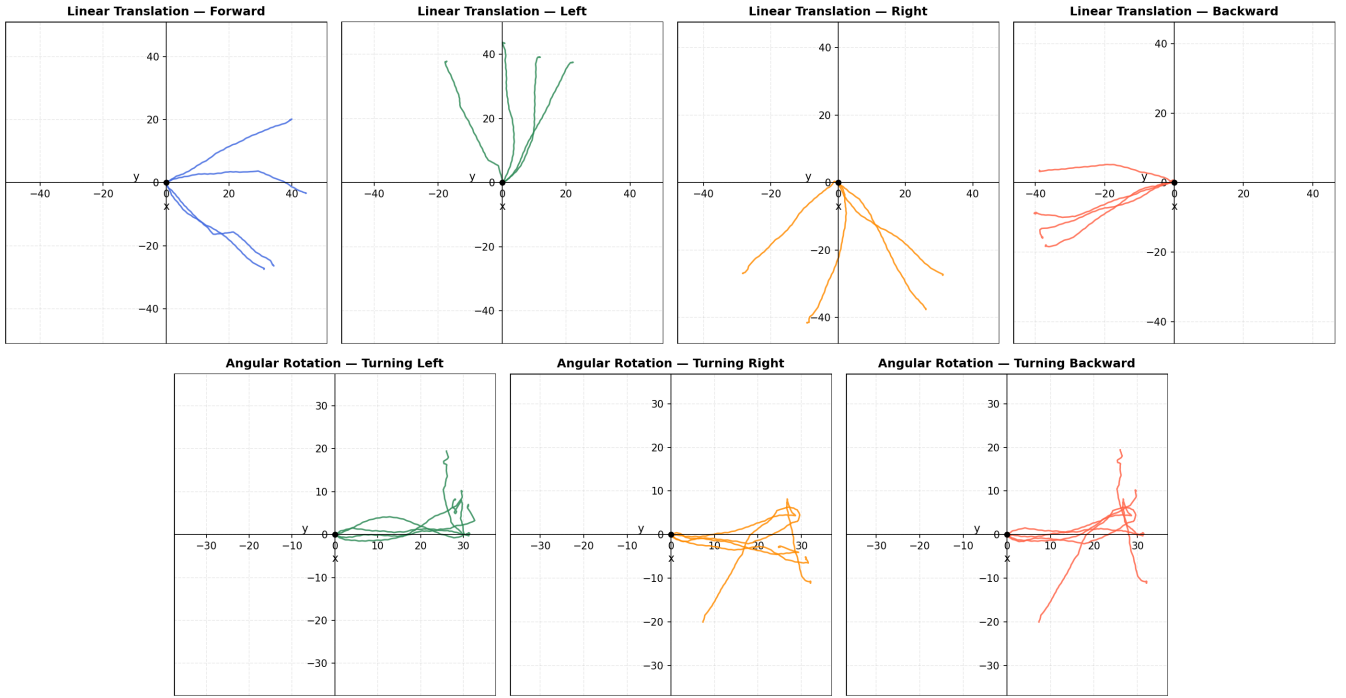


Fig. 1: Cartesian trajectory visualizations of discovered skills on the MuJoCo Ant. All paths originate from the origin  $(0, 0)$ . The top row trajectories correspond to linear translation skills, separated into distinct directional quadrants. The bottom row trajectories correspond to angular rotation skills, demonstrating isolated turning behaviors.

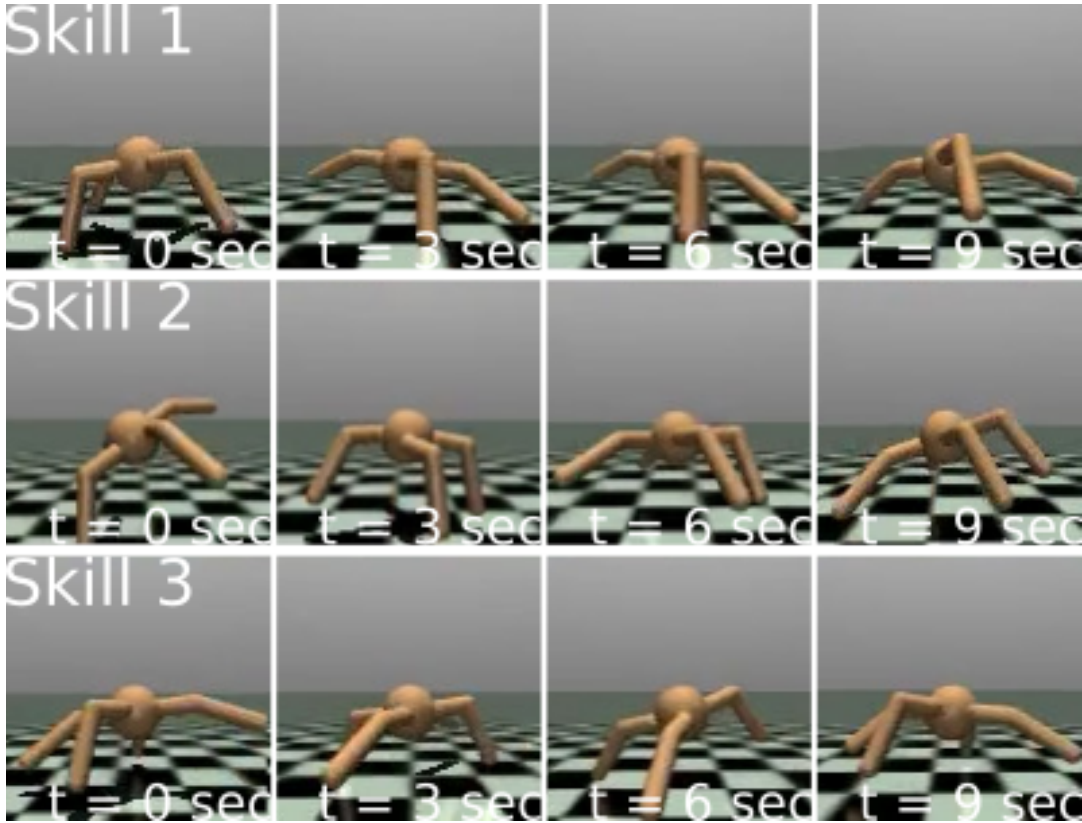


Fig. 2: Qualitative results of DDMF discovered skills on the Ant robot. Each row represents a time-lapse rollout, from  $t = 0$  to  $t = 9$  seconds, conditioned on a distinctly sampled latent skill. DDMF demonstrates its capability of discovering diverse, physically viable actions for locomotion skills, while maintaining a stable, upright posture.

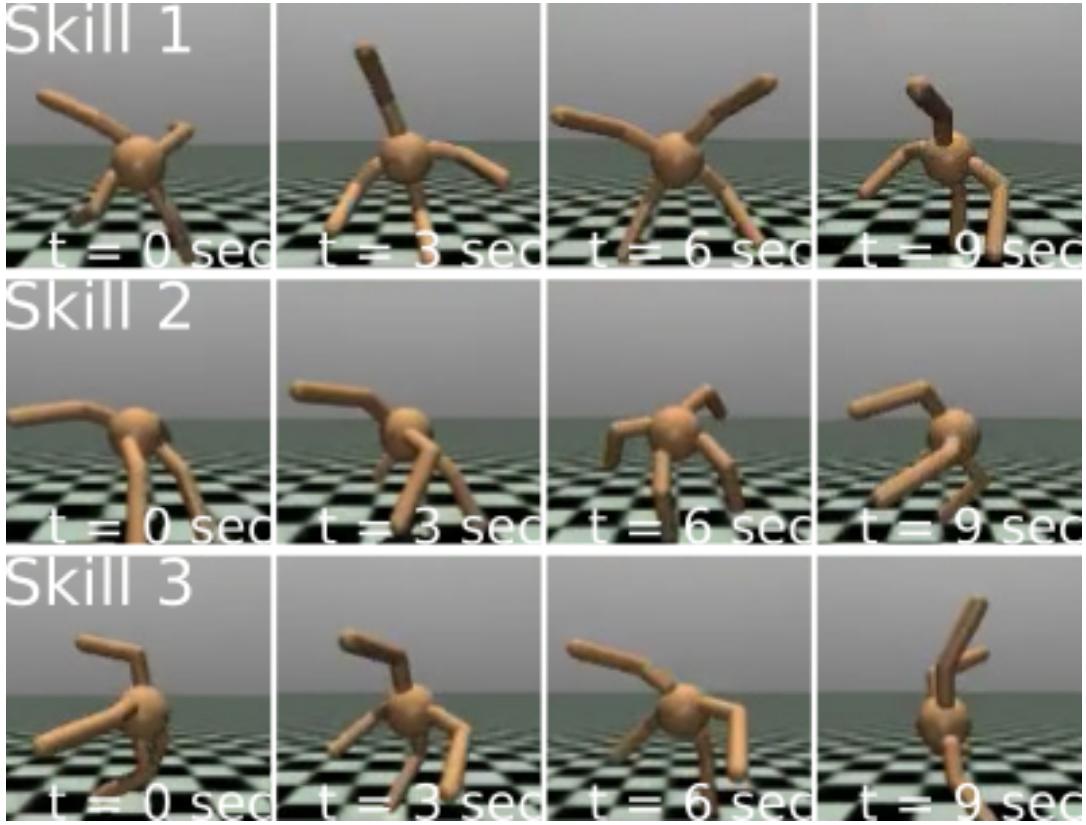


Fig. 3: Qualitative results of the SUSD baseline on the Ant robot. Each row represents a time-lapse rollout, from  $t = 0$  to  $t = 9$  seconds, conditioned on a randomly sampled latent skill. The SUSD baseline exhibits severe mode collapse and physically infeasible behaviors. Across all sampled skills, the robot fails to execute stable quadrupedal locomotion, instead tumbling and “cartwheeling” to achieve displacement.

important to note that the high-level conditional flow policy is not deployed directly on the physical hardware for closed-loop control. Instead, we utilize a sim-to-real trajectory tracking approach.

*a) Simulation Trajectory Generation:* The hardware deployment acts as a physical verification of the semantic state factorization defined for the locomotion task. First, the trained DDMF policy is executed within the MuJoCo simulation environment using an Ant robot. By conditioning the policy on a specific discovered latent skill  $z$ , the robot rolls out an episode in simulation. During this rollout, we extract and record the resulting kinematic trajectory, specifically isolating the Cartesian path and heading commands generated from the latent skill.

*b) Hardware Trajectory Tracking:* The pre-computed simulated trajectories are then transferred to the physical Unitree Go2. Specifically, we use the physical robot as a path follower. The extracted waypoints and the angular rotations are fed into the Go2’s internal locomotion controller via the Unitree SDK 2.0 through an ethernet cable. The onboard controller, running at the frequency of 50 Hz, handles the low-level foot placement, balancing, and joint-level proportional-derivative tracking required to replicate the simulated Ant’s behavior in the physical world.

*c) Verification of Skill Transfer:* Our setup explicitly isolates and tests the quality of the DDMF’s high-level semantic skill disentanglement. The hardware trial demonstrates that the latent skill translates to physically viable paths, including linear translation and angular rotation, that a standard quadruped can track without failure.

## V. ADDITIONAL EXPERIMENT RESULTS

In this section, we provide additional experimental results within the MuJoCo Ant locomotion environment. We first analyze the Cartesian trajectories of the locomotion skills discovered by our DDMF approach to demonstrate its skill disentanglement. Following this, we present extended qualitative results that compare the behavioral diversity and physical stability of the robot actions learned by our approach against baseline methods.

### A. Visualization of Locomotion Trajectories in Ant Locomotion Task

We visualize the two-dimensional Cartesian trajectories executed by the Ant robot across a variety of discovered skills. Figure 1 plots the  $(x, y)$  coordinate paths over a fixed evaluation horizon, demonstrating how distinct latent skill map to interpretable locomotion behaviors.

a) *Linear Translation Skills*: The top row of Figure 1 illustrates trajectories generated by skills associated with linear translation. The discovered skills partition the robot locomotion behaviors into four categories: walking forward, left, right, and backward ( $-x$ ). In addition, for each skill, we can observe a diverse set of trajectories. This intra-skill diversity confirms that the diversified flow policy does not merely memorize a single deterministic path, but captures a rich distribution of valid trajectories.

b) *Angular Rotation Skills*: The bottom row of Figure 1 displays trajectories associated with angular rotation factors. The behaviors within rotation skills demonstrate distinct turning maneuvers, including turning left, turning right, and turning backward.

## B. Qualitative Results in Ant Locomotion Task

We provide an extended qualitative analysis in Ant locomotion environment to compare the locomotion skills learned by the DDMF approach against the SUSD baseline.

a) *Stable and Diverse Locomotion in DDMF*: We first illustrate the behaviors discovered by our DDMF approach on the Ant robot in Figure 2. During evaluation, we randomly sample three distinct latent skills. Across all sampled skills, the flow policy generates robust locomotion actions where the Ant consistently maintains a stable, upright posture while exhibiting distinct and diverse leg actuation patterns. By leveraging the highly expressive capacity of the conditional flow matching policy, DDMF successfully discovers a repertoire of physically viable and diverse robot behaviors.

b) *Mode Collapse of Behavioral Diversity in SUSD*: In contrast, the behaviors discovered by the SUSD baseline, illustrated in Fig. 3, exhibit a mode collapse in behavioral diversity and physically infeasible actions. Regardless of the sampled latent skill, the SUSD approach fails to execute stable quadrupedal locomotion. Instead, the policy causes the Ant to tumble or “cartwheel” to achieve rotation or displacement. This type of behavior proves that conventional disentangled skill discovery methods lack the representational capacity to effectively model multi-modal action diversity in continuous control tasks.

## REFERENCES

- [1] S. Ozair, C. Lynch, Y. Bengio, A. Van den Oord, S. Levine, and P. Sermanet, “Wasserstein dependency measure for representation learning,” *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [2] S. M. H. Hosseini and M. S. Baghshah, “SUSD: Structured unsupervised skill discovery through state factorization,” in *The Fourteenth International Conference on Learning Representations*, 2026.
- [3] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, “Neural ordinary differential equations,” *Advances in neural information processing systems*, vol. 31, 2018.
- [4] S. Yang, B. Liu, and H. Chen, “Policyflow: Policy optimization with continuous normalizing flow in reinforcement learning,” *arXiv preprint arXiv:2602.01156*, 2026.
- [5] A. Gupta, V. Kumar, C. Lynch, S. Levine, and K. Hausman, “Relay policy learning: Solving long-horizon tasks via imitation and reinforcement learning,” in *Conference on Robot Learning*, 2020, pp. 1025–1037.
- [6] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.

- [7] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ international conference on intelligent robots and systems*, 2012, pp. 5026–5033.
- [8] C. Eastwood and C. K. I. Williams, “A framework for the quantitative evaluation of disentangled representations,” in *International Conference on Learning Representations*, 2018.
- [9] P. Prettenhofer and G. Louppe, “Gradient boosted regression trees in scikit-learn,” in *PyData 2014*, 2014.